

Pin Mapping & Connections

AVR128DA48 — LCD & Touch Keypad Interface

Field	Detail
Microcontroller	Microchip AVR128DA48 (48-pin), internal HF oscillator @ 24 MHz
Display	Clover Display S5026 — 4-digit, 7-segment LCD, direct 2-wire serial drive
Keypad	TTP229 capacitive touch controller — 8-key serial output mode
Prepared by	Dwij Vyas, Roshan Mali

1. Overview

This document records the complete pin-level interface between the AVR128DA48 and its two peripherals: the Clover S5026 4-digit LCD and the TTP229 touch keypad.

Both peripherals are driven by bit-banged GPIO — there is no I²C, SPI, or UART peripheral involved in this design. Each uses a proprietary serial protocol rather than a standard bus, so the bit-level framing is documented alongside the pin assignments below.

2. Pin Mapping Summary

Consolidated view of every MCU pin used in this design.

Pin	Signal	Peripheral	Function	Direction	Pull-up
PC0	LCLK	Clover S5026 LCD	Serial clock for display frame	Output	—
PC1	LDATA	Clover S5026 LCD	Serial data for display frame	Output	—
PD0	SCL	TTP229 keypad	Scan clock (idle high)	Output	—
PD1	SDO	TTP229 keypad	Key data out (open-drain, active low)	Input	Internal

3. LCD Interface — Clover S5026 Display

3.1 Connections

Pin	Signal	Description
PC0	LCLK	Clock line. Display samples LDATA on the rising edge of LCLK.
PC1	LDATA	Serial data line, set before each rising edge of LCLK.

3.2 Frame Protocol

Each update is a 37-bit serial frame, sent bit-by-bit with no use of a standard bus protocol:

- Header — 3 bits: 1, 0, 1
- Data — 32 bits, one per segment electrode, LSB first
- Trailer — 2 bits: 0, 0

Per-bit timing: data line set → 100 µs setup → LCLK high for 100 µs → LCLK low for 100 µs (300 µs/bit, ≈11.1 ms per full frame).

The display has no persistent latch on the MCU side — frames must be retransmitted continuously (every ~20–30 ms in firmware) or the digits fade out within a few seconds.

3.3 Segment Bit Map

Bit index (0–31) within the 32-bit data field, by digit position and segment:

Segment	Digit 1 (rightmost)	Digit 2	Digit 3	Digit 4 (leftmost)
A	13	28	24	20
B	0	29	25	21
C	2	6	10	14
D	3	7	11	15
E	4	8	12	16
F	31	27	23	19
G	30	26	22	18

Bit positions 1, 5, 9, and 17 are not driven by current firmware. Their function on the display glass (icon, colon, sign segment, or unused) has not been confirmed — see Section 6.

4. Keypad Interface — TTP229 Touch Controller

4.1 Connections

Pin	Signal	Description
PD0	SCL	Scan clock, MCU-driven, idle high.
PD1	SDO	Open-drain key data output from the TTP229; internal pull-up enabled on this pin.

4.2 Read Protocol

8-key serial output mode. To scan: pull SCL low, wait $\geq 10\ \mu\text{s}$ for SDO to settle, sample SDO (active low = touched), return SCL high. Up to 8 clock cycles are issued per scan; the loop index of the first active-low result (1–8) is returned as the key number, or 0 if no key is touched.

This is a capacitive touch controller, not a mechanical matrix keypad — no debounce is required at the electrical level, though the firmware still confirms a press over two consecutive scans before acting on it.

5. Firmware Reference

5.1 Description

The listing below implements the LCD and keypad interfaces described in Sections 3 and 4. On each TTP229 touch, the firmware confirms the key over two consecutive scan cycles before accepting it, then drives that digit (1–8) identically across all four display positions.

The display frame is retransmitted unconditionally on every loop pass (~20–30 ms), whether or not the digit has changed. This is required because the display holds no state of its own — without continuous refresh, the image fades out within a few seconds. Key debounce is implemented as a non-blocking state machine specifically so that it never stalls this refresh loop.

- Non-blocking debounce — a press is confirmed after 2 consecutive matching scans, then latched until release.
- Unconditional continuous refresh — `SEND_FRAME()` runs every loop iteration without exception.

- All four digit positions mirror the same value — this firmware does not implement independent multi-digit entry.

5.2 Source Listing

```
#define F_CPU 24000000UL

#include <avr/io.h>
#include <avr/cpufunc.h>
#include <util/delay.h>
#include <stdint.h>

// ===== LCD CONNECTIONS =====

#define LCD_CLK_PIN    PIN0_bm    // PC0 -> LCLK
#define LCD_DATA_PIN   PIN1_bm    // PC1 -> LDATA

// ===== TTP229 CONNECTIONS =====

#define TTP_SCL_PIN    PIN0_bm    // PD0 -> SCL
#define TTP_SDO_PIN    PIN1_bm    // PD1 -> SDO

#define BIT(x) (1UL << (x))

// ===== SEGMENT MAP =====

// Digit 1 (rightmost)
#define SEG_1A    13
#define SEG_1B    0
#define SEG_1C    2
#define SEG_1D    3
#define SEG_1E    4
#define SEG_1F    31
#define SEG_1G    30

// Digit 2
#define SEG_2A    28
#define SEG_2B    29
#define SEG_2C    6
#define SEG_2D    7
#define SEG_2E    8
#define SEG_2F    27
#define SEG_2G    26

// Digit 3
#define SEG_3A    24
#define SEG_3B    25
#define SEG_3C    10
#define SEG_3D    11
#define SEG_3E    12
#define SEG_3F    23
#define SEG_3G    22

// Digit 4 (leftmost)
#define SEG_4A    20
#define SEG_4B    21
#define SEG_4C    14
#define SEG_4D    15
#define SEG_4E    16
#define SEG_4F    19
```

```
#define SEG_4G    18

// ===== LCD FUNCTIONS =====

static inline void DATA_L(uint8_t bit)
{
    if (bit)
        PORTC.OUTSET = LCD_DATA_PIN;
    else
        PORTC.OUTCLR = LCD_DATA_PIN;

    _delay_us(100);

    PORTC.OUTSET = LCD_CLK_PIN;
    _delay_us(100);

    PORTC.OUTCLR = LCD_CLK_PIN;
    _delay_us(100);
}

static void SEND_FRAME(uint32_t value)
{
    DATA_L(1);

    DATA_L(0);
    DATA_L(1);

    for (uint8_t i = 0; i < 32; i++)
    {
        DATA_L((value >> i) & 1);
    }

    DATA_L(0);
    DATA_L(0);
}

// ===== TTP229 READ =====

static uint8_t ttp229_read(void)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        PORTD.OUTCLR = TTP_SCL_PIN;
        _delay_us(10);

        if (!(PORTD.IN & TTP_SDO_PIN))
        {
            PORTD.OUTSET = TTP_SCL_PIN;
            return i + 1;
        }

        PORTD.OUTSET = TTP_SCL_PIN;
        _delay_us(10);
    }

    return 0;
}

// ===== DIGIT GENERATOR =====
```

```

#define DIGIT_FUNC(name,A,B,C,D,E,F,G) \
static uint32_t name(uint8_t num) \
{ \
    switch (num) \
    { \
        case 1: return BIT(B) | BIT(C); \
        case 2: return BIT(A)|BIT(B)|BIT(D)|BIT(E)|BIT(G);\
        case 3: return BIT(A)|BIT(B)|BIT(C)|BIT(D)|BIT(G);\
        case 4: return BIT(B)|BIT(C)|BIT(F)|BIT(G); \
        case 5: return BIT(A)|BIT(C)|BIT(D)|BIT(F)|BIT(G);\
        case 6: return BIT(A)|BIT(C)|BIT(D)|BIT(E)|BIT(F)|BIT(G);\
        case 7: return BIT(A)|BIT(B)|BIT(C); \
        case 8: return BIT(A)|BIT(B)|BIT(C)|BIT(D)|BIT(E)|BIT(F)|BIT(G);\
        default: return 0; \
    } \
}

DIGIT_FUNC(digit1, SEG_1A, SEG_1B, SEG_1C, SEG_1D, SEG_1E, SEG_1F, SEG_1G)
DIGIT_FUNC(digit2, SEG_2A, SEG_2B, SEG_2C, SEG_2D, SEG_2E, SEG_2F, SEG_2G)
DIGIT_FUNC(digit3, SEG_3A, SEG_3B, SEG_3C, SEG_3D, SEG_3E, SEG_3F, SEG_3G)
DIGIT_FUNC(digit4, SEG_4A, SEG_4B, SEG_4C, SEG_4D, SEG_4E, SEG_4F, SEG_4G)

// ===== MAIN =====

int main(void)
{
    _PROTECTED_WRITE(CLKCTRL.OSCHFCTRLA,
                     CLKCTRL_FRQSEL_24M_gc);

    PORTC.DIRSET = LCD_CLK_PIN | LCD_DATA_PIN;
    PORTC.OUTCLR = LCD_CLK_PIN | LCD_DATA_PIN;

    PORTD.DIRSET = TTP_SCL_PIN;
    PORTD.DIRCLR = TTP_SDO_PIN;

    PORTD.OUTSET = TTP_SCL_PIN;
    PORTD.PIN1CTRL = PORT_PULLUPEN_bm;

    uint8_t current_digit = 0;
    uint8_t candidate_key = 0; // key value we're currently sampling for confirmation
    uint8_t candidate_count = 0; // how many consecutive scans have agreed on candidate_key
    uint8_t key_active = 0; // a press has already been confirmed & acted on - ignore until release

    while (1)
    {
        uint8_t key = ttp229_read();

        // ---- Non-blocking debounce / press latch -----
        // No delays here at all - the display refresh below must never be
        // interrupted, since gaps in the refresh stream are what showed up
        // as glitches during transitions. A press is confirmed once the
        // same key has been read on 2 consecutive scans (~20-30ms apart,
        // the natural loop cadence), and then latched so it can't
        // re-trigger until the key is released.
        if (key == 0)
        {
            candidate_key = 0;
            candidate_count = 0;
            key_active = 0;
        }
    }
}

```

```
else if (!key_active)
{
    if (key == candidate_key)
    {
        candidate_count++;
    }
    else
    {
        candidate_key = key;
        candidate_count = 1;
    }

    if (candidate_count >= 2)
    {
        current_digit = key;
        key_active = 1;
    }
}
// else: press already confirmed and latched - ignore further reads
// until release.

// ---- Continuous refresh -----
// Runs unconditionally every loop, at an even ~20-30ms cadence,
// with a direct in-place pattern swap on digit change. No blanking
// step: for a binary on/off segment display, the smoothest
// possible transition is an instant swap inside an unbroken,
// evenly-paced refresh stream - any deliberate blank/clear frame
// is itself a visible flicker, not a smoothing technique.
uint32_t pattern =
    digit1(current_digit)
    | digit2(current_digit)
    | digit3(current_digit)
    | digit4(current_digit);

SEND_FRAME(pattern);

_delay_ms(20);
}
}
```

6. Items Requiring Hardware Verification

- LCD segment bits 1, 5, 9, and 17 — currently undriven. Confirm whether any of these correspond to a usable sign, colon, or icon segment on the display glass.